LAPORAN PRAKTIKUM
Sistem Operasi

MODUL 10
(Shell)

Disusun Oleh:
Ryan Gabriel Togar Simamora (2311104045)
Kelas S1SE-07-02

Dosen : Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I
PENDAHULUAN

1.1 Dasar Teori

A. Shell

Shell adalah antarmuka berbasis teks yang memungkinkan pengguna berinteraksi dengan sistem operasi. Pengguna cukup mengetikkan perintah (misalnya echo, ping, atau uptime), lalu shell akan menerjemahkannya menjadi eksekusi fungsi kernel yang sesuai, tanpa perlu memanggil system call secara manual.

1. **Karakteristik Shell pada Xinu**
   Shell di Xinu memiliki ciri khas sebagai berikut:
   - Internal: Semua perintah berupa fungsi C internal
   - Terintegrasi: Shell dan kernel berada dalam satu kesatuan
   - Statis: Penambahan perintah memerlukan modifikasi kode dan kompilasi ulang sistem

2. **Alur Kerja Shell (Read — Parse — Execute)**
   Shell bekerja dalam loop yang terus berjalan dengan tiga tahap utama:

   a. Read (Membaca Input)
   Shell menampilkan prompt (misalnya xsh $) dan menunggu pengguna memasukkan perintah hingga menekan ENTER.

   b. Parse (Memproses Input)
   Input dipecah menjadi beberapa bagian:
   - **Token pertama** → nama perintah
   - **Token berikutnya** → argumen

   ```
   ls -al
   ```

   Perintah: ls
   Argumen: -al

3. **Execute (Menjalankan Perintah)**
   Shell mencari fungsi internal yang sesuai dengan nama perintah. Jika ditemukan, fungsi tersebut dieksekusi dengan argumen yang telah diproses.

4. Perbedaan Eksekusi Perintah: Xinu vs Linux
Perbedaan paling mendasar antara Xinu dan sistem operasi umum seperti Linux terdapat pada cara perintah disimpan dan dijalankan:

Linux (Program Eksternal)
Di Linux, perintah shell seperti ls, cat, atau grep merupakan program yang berdiri sendiri. Program tersebut disimpan sebagai file executable dalam filesystem (misalnya di /bin/ls).

Xinu (Fungsi Internal)
Di Xinu, perintah shell bukan program terpisah. Setiap perintah adalah fungsi C yang sudah berada di memori. Fungsi seperti xsh_echo, xsh_kill, xsh_ps dikompilasi dan di-link menjadi satu dengan kernel dan shell, membentuk satu image sistem operasi utuh.

Implikasinya: Karena semua perintah menjadi bagian dari sistem, menambah atau menghapus perintah membutuhkan perubahan kode sumber dan kompilasi ulang seluruh sistem.

Tujuan Praktikum:

1. Mahasiswa mampu memodifikasi shell pada Xinu.

Manfaat Praktikum :

1. Memahami Shell

BAB II
HASIL PRAKTIKUM

2.1 Guided Praktikum

A. Persiapan Implementasi Dalam Xinu

Kita akan melakukan proses registrasi, deklarasi, dan implementasi untuk menambahkan perintah baru yang bernama “mycmd”. Tahap ini dilakukan untuk mendaftarkan mycmd ke shell.c, sehingga Shell mengenalinya sebagai perintah valid yang dapat dijalankan. Registrasi juga membuat mycmd muncul dalam daftar perintah yang tersedia ketika pengguna mengeceknya di terminal.

B. Persiapan Praktikum

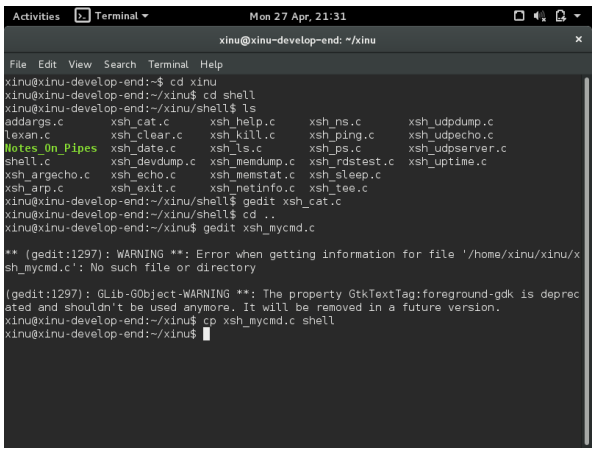
1. Registrasi Command ke Shell

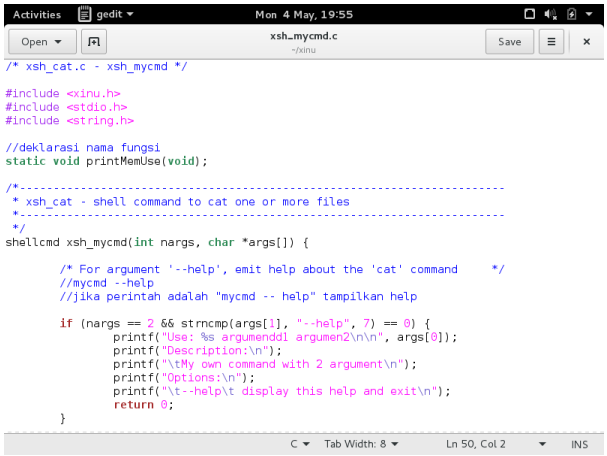
Untuk menambahkan perintah baru bernama **mycmd**, dilakukan tiga tahap utama:

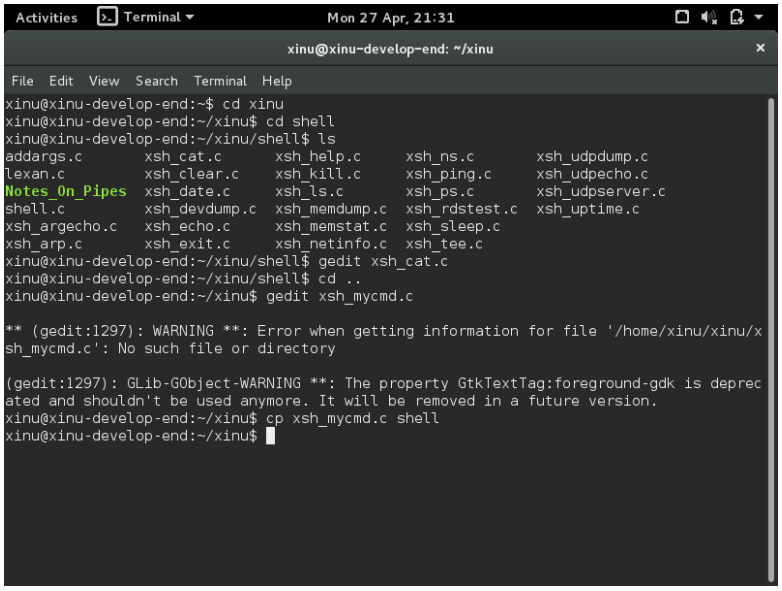
1. **Registrasi** → mendaftarkan perintah ke shell.c
2. **Deklarasi** → mendefinisikan fungsi perintah
3. **Implementasi** → menuliskan logika perintah

Tahap registrasi memastikan mycmd dikenali oleh shell dan muncul sebagai perintah yang valid di terminal.

Disini kita buat dulu file xsh_mycmd.c dengan perintah gedit dan modifikasi isi filenya sesuai gambar berikut :





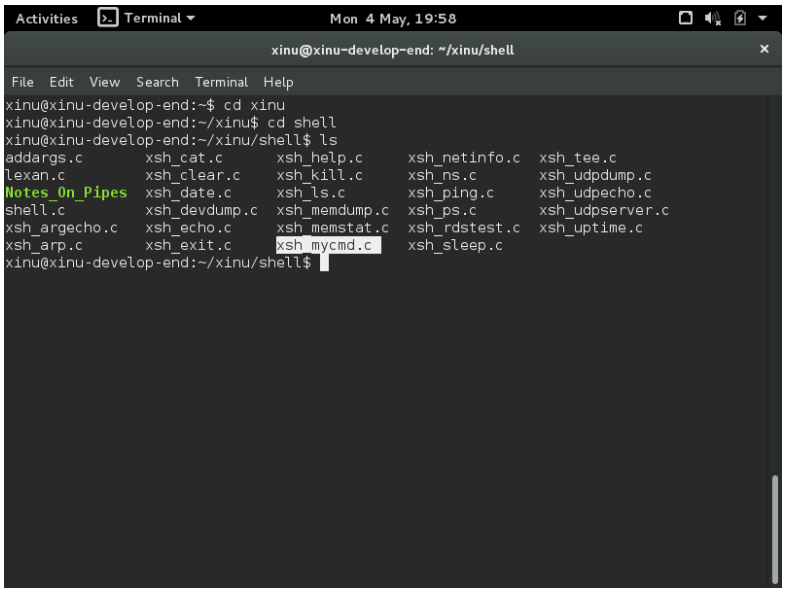


langkah selanjutnya adalah menyalinnya ke dalam direktori sistem shell dan memastikan file tersebut tersimpan dengan benar.

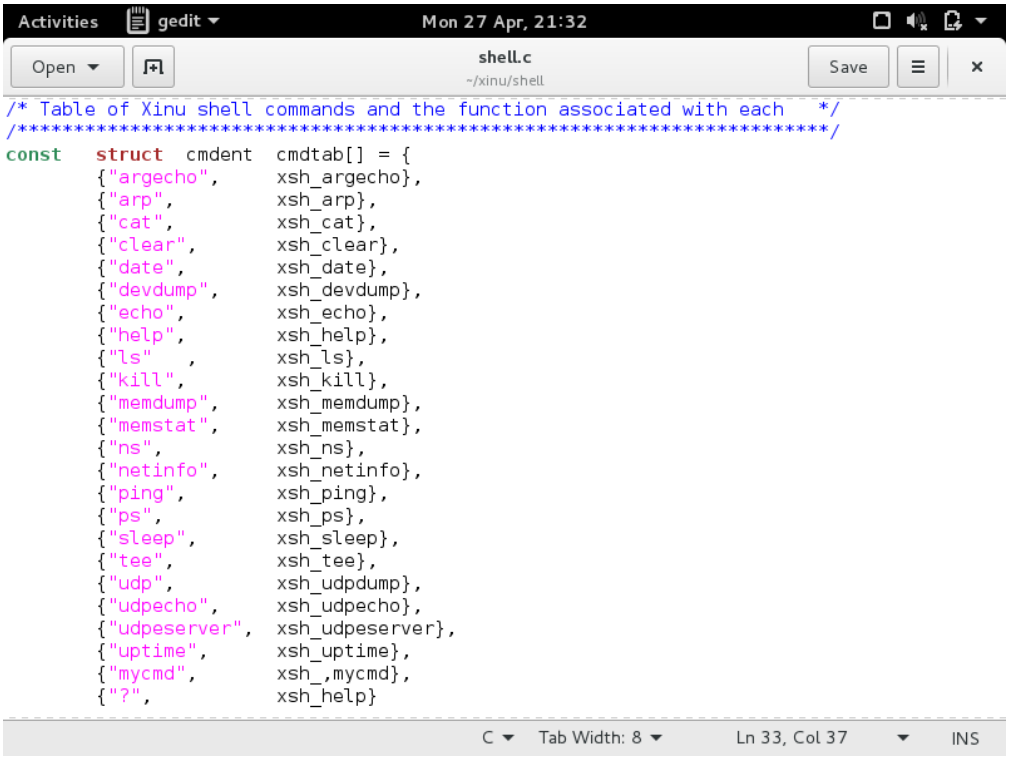
Langkah Implementasi:

1. Salin file ke direktori shell: cp xsh_mycmd.c shell
2. Masuk ke direktori shell: cd shell
3. Tampilkan daftar file untuk verifikasi: ls

Jika langkah ini sukses, Anda akan melihat xsh_mycmd.c muncul dalam daftar file yang ditampilkan.



2. Kemudian tambahkan dengan perintah {"mycmd", xsh_mycmd} dan save.



3. Deklarasi Perintah

Agar fungsi `xsh_mycmd` dapat dikenali dan dipanggil oleh program utama Shell (di `shell.c`), kita perlu mendeklarasikannya secara global. Hal ini dilakukan dengan menambahkan prototipe fungsi menggunakan kata kunci `extern` di dalam file header `shprototypes.h`.

Dari direktori `shell` saat ini, kita akan masuk ke direktori `include` dan buka file tersebut menggunakan perintah berikut:

```
cd ../include gedit shprototypes.h
```

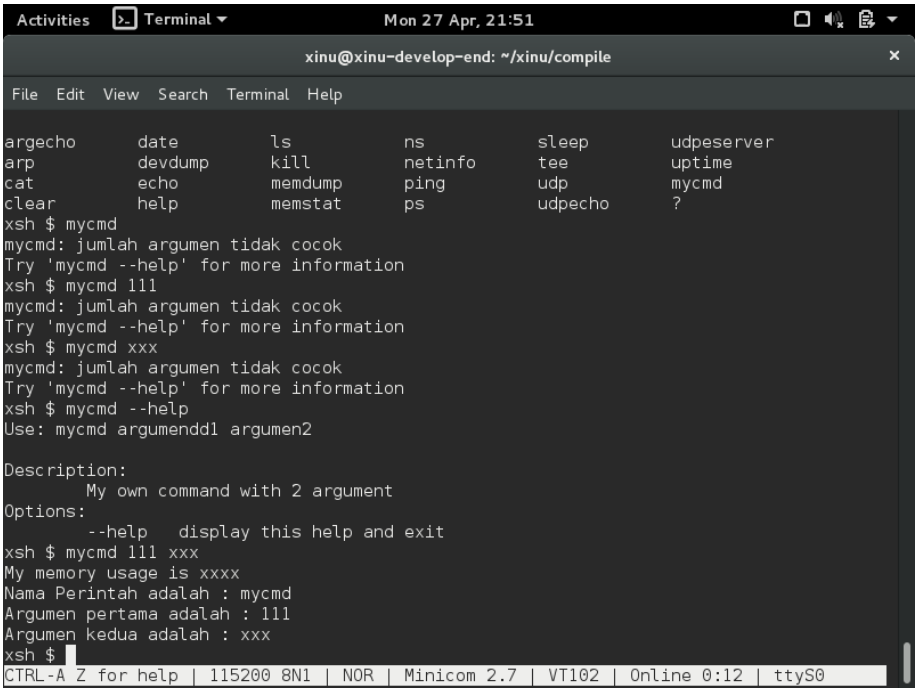
Cari baris yang memuat extern shellcmd xsh_help. Tepat di bawah baris tersebut, tambahkan kode deklarasi berikut:

extern shellcmd xsh_mycmd (int32, char *[]);

Pastikan penempatannya sesuai dengan referensi gambar, kemudian simpan (Save) perubahan pada file tersebut.

4. Setelah semua konfigurasi selesai, lakukan pembersihan dan kompilasi ulang kernel menggunakan perintah make clean dan make. Jika proses kompilasi sukses, silakan buka terminal serial dengan mengetik sudo minicom (masukkan password saat diminta), lalu segera aktifkan backend pada VirtualBox.

   Untuk memverifikasi apakah fungsi berjalan dengan benar, lakukan pengujian dengan mengetikkan perintah mycmd (tanpa argumen), dilanjutkan dengan mycmd 111 (satu argumen), dan terakhir mycmd 111 xxx (dua argumen), lalu tekan Enter pada setiap perintah untuk mengamati bagaimana sistem merespons variasi input tersebut.



2.2 Jurnal Praktikum

Jurnal

1. **[40 Poin]** Akan dimodifikasi shell dengan modifikasi syscall bernama **chname** yang berfungsi untuk mengubah nama suatu proses. Lihat kembali modul sebelumnya cara membuat syscall.

   Perhatikan sekarang syscall **chname** mempunyai 3 parameter yaitu pid, character dan panjang character. Character untuk menyimpan nama dan panjang character untuk panjang nama.

   a. Pada prototypes.h chname diubah menjadi:

```
/* in file chname.c */

extern  pri16   chname(pid32, char *, uint32);
```

   Jawab :

   b. Pada chname.c fungsi diubah dari:

```
pri16   chname(
          pid32         pid,            /* ID of process to change      */
          pri16         newprio         /* New priority                 */
        )
```

   menjadi

```
pri16   chname(
          pid32         pid,            /* ID of process to change      */
          char          *newname,
          uint32        namelen
        )
```

   Modifikasi kode pada chname.c sehingga nama proses bisa diubah bila syscall tersebut dipanggil.

Jawab :

Masuk ke file chname.c yang berada di direktori system dengan perintah gedit chname.c

```
Activities  Terminal  Mon 4 May, 22:25
xinu@xinu-develop-end: ~/xinu/system
File  Edit  View  Search  Terminal  Help
ctype.h     ethloop.h  ip.h         pci.h         ramdisk.h     stddef.h        xinu.h
date.h      file.h     kernel.h     pipe.h        rdisksys.h    stdio.h
debug.h     flash.h    lfilesys.h   ports.h       resched.h     stdlib.h
delay.h     i386.h     limits.h     process.h     rfilesys.h    string.h
device.h    icmp.h     mark.h       prototypes.h  sdmc.h        testsuite.h
xinu@xinu-develop-end:~/xinu/include$ gedit shprototypes.h
xinu@xinu-develop-end:~/xinu/include$ gedit prototypes.h
xinu@xinu-develop-end:~/xinu/include$ gedit shprototypes.h
xinu@xinu-develop-end:~/xinu/include$ cd ..
xinu@xinu-develop-end:~/xinu$ cd system
xinu@xinu-develop-end:~/xinu/system$ ls
ascdate.c     DESCRIPTION  getutime.c    meminit.c    queue.c      signaln.c
bufinit.c     device       i386.c        mkbufpool.c  read.c       sleep.c
chname.c      evec.c       include       net          ready.c      stacktrace.c
chprio.c      exit.c       init.c        newqueue.c   receive.c    start.S
clkdisp.S     freebuf.c    initialize.c  open.c       recvclr.c    suspend.c
clkhandler.c  freemem.c    insert.c      panic.c      recvtime.c   system
clkinit.c     getbuf.c     insertd.c     pci.c        resched.c    unsleep.c
close.c       getc.c       intr.S        ptclear.c    resume.c     userret.c
compile       getdev.c     ioerr.c       ptcount.c    seek.c       wait.c
conf.c        getitem.c    ionull.c      ptcreate.c   semcount.c   wakeup.c
config        getmem.c     kill.c        ptdelete.c   semcreate.c  write.c
control.c     getpid.c     kprintf.c     ptinit.c     semdelete.c  xdone.c
COPYRIGHT     getprio.c    lib           ptrecv.c     semreset.c   yield.c
create.c      getstk.c     main.c        ptreset.c    send.c
ctxsw.S       getticks.c   mark.c        ptsend.c     shell
debug.c       gettime.c    memdemo.c     putc.c       signal.c
xinu@xinu-develop-end:~/xinu/system$ gedit chname.c
```

Edit File sesuai yang diminta pada soal

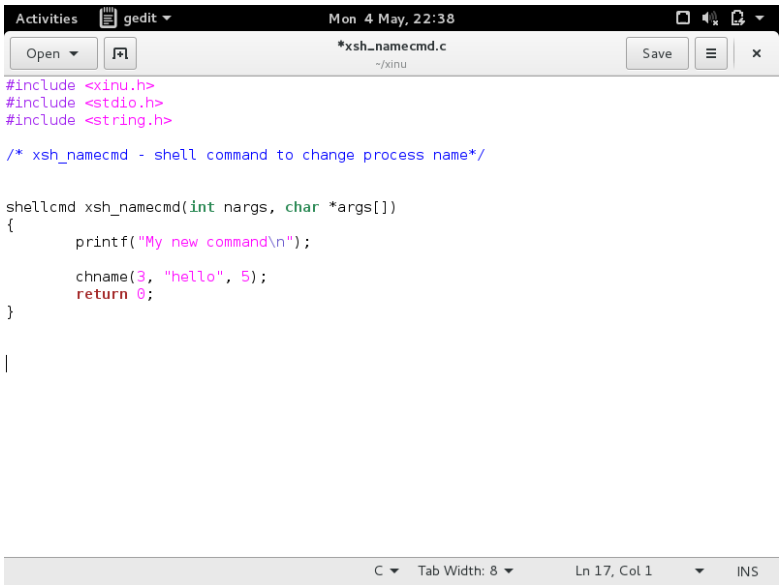
```
Activities  gedit  Tue 5 May, 00:01
Open   chname.c  ~/xinu/system   Save
#include <xinu.h>

/*------------------------------------------------------------------------
 *  chname  - Dummy syscall
 *------------------------------------------------------------------------
 */
pri16   chname(
          pid32         pid,            /* ID of process to change      */
          char          *newname,
          uint32        namelen
        )
{
        intmask mask;                   /* Saved interrupt mask         */
        struct  procent *prptr;         /* Ptr to process's table entry */

        mask = disable();
        if (isbadpid(pid)) {
                restore(mask);
                return (pri16) SYSERR;
        }
        prptr = &proctab[pid];
        strncpy(prptr->prname, newname, namelen);
        prptr->prname[namelen] = '\0';

        restore(mask);
        return OK;
}
C  Tab Width: 8  Ln 29, Col 2  INS
```

2. **[40 Poin]** Buatlah perintah baru bernama namecmd sesuai dengan langkah-langkah pada no.5 pada modul shell!
   Berikut adalah kode dalam perintah baru namecmd:

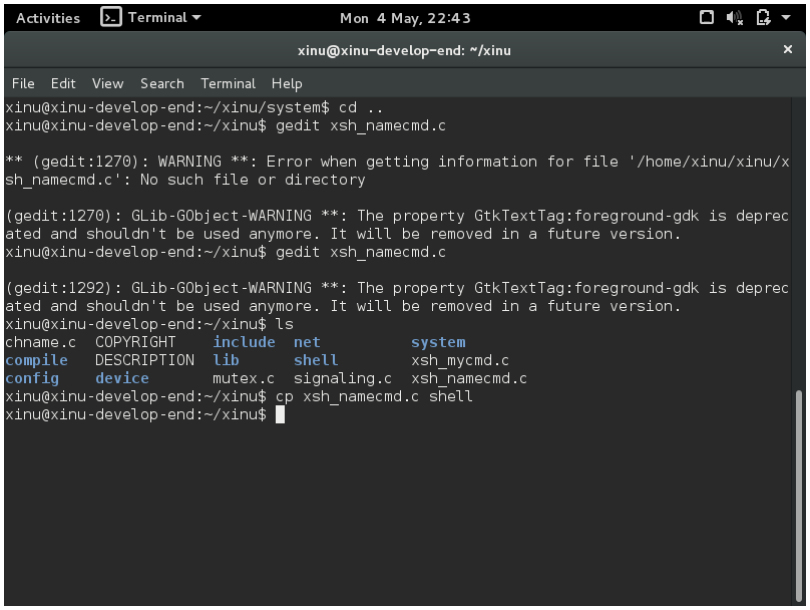
```
printf("My new command");

// pid 3 = netin; mengubah nama proses pid 3 yaitu netin menjadi hello
chname(3,"hello",5);

return 0;
```

Jawab :

Buat file xsh_namecmd.c di direktori xinu dengan perintah gedit xsh_namecmd.c , lalu save dan exit



Lalu copy file yang baru kita modifikasi ke direktori shell dengan perintah cp xsh_namecmd.c shell



Terlihat file xsh_namecmd.c sudah berada di direktori shell

```
Activities  Terminal ▾        Mon 4 May, 22:46
xinu@xinu-develop-end: ~/xinu/shell
File  Edit  View  Search  Terminal  Help
xinu@xinu-develop-end:~/xinu/system$ cd ..
xinu@xinu-develop-end:~/xinu$ gedit xsh_namecmd.c

** (gedit:1270): WARNING **: Error when getting information for file '/home/xinu/xinu/x
sh_namecmd.c': No such file or directory

(gedit:1270): GLib-GObject-WARNING **: The property GtkTextTag:foreground-gdk is deprec
ated and shouldn't be used anymore. It will be removed in a future version.
xinu@xinu-develop-end:~/xinu$ gedit xsh_namecmd.c

(gedit:1292): GLib-GObject-WARNING **: The property GtkTextTag:foreground-gdk is deprec
ated and shouldn't be used anymore. It will be removed in a future version.
xinu@xinu-develop-end:~/xinu$ ls
chname.c   COPYRIGHT    include  net          system
compile    DESCRIPTION  lib      shell        xsh_mycmd.c
config     device       mutex.c  signaling.c  xsh_namecmd.c
xinu@xinu-develop-end:~/xinu$ cp xsh_namecmd.c shell
xinu@xinu-develop-end:~/xinu$ cd shell
xinu@xinu-develop-end:~/xinu/shell$ ls
addargs.c        xsh_cat.c      xsh_help.c     xsh_namecmd.c  xsh_sleep.c
lexan.c          xsh_clear.c    xsh_kill.c     xsh_netinfo.c  xsh_tee.c
Notes_On_Pipes   xsh_date.c     xsh_ls.c       xsh_ns.c       xsh_udpdump.c
shell.c          xsh_devdump.c  xsh_memdump.c  xsh_ping.c     xsh_udpecho.c
xsh_argecho.c    xsh_echo.c     xsh_memstat.c  xsh_ps.c       xsh_udpserver.c
xsh_arp.c        xsh_exit.c     xsh_mycmd.c    xsh_rdstest.c  xsh_uptime.c
xinu@xinu-develop-end:~/xinu/shell$ gedit xsh_namecmd.c
xinu@xinu-develop-end:~/xinu/shell$
```

Lalu kita Registrasi: Tambahkan ke shell.c dan save

```
Activities  gedit ▾        Mon 4 May, 22:49
Open ▾   *shell.c  ~/xinu/shell   Save
        {"kill",        xsh_kill},
        {"memdump",     xsh_memdump},
        {"memstat",     xsh_memstat},
        {"ns",          xsh_ns},
        {"netinfo",     xsh_netinfo},
        {"ping",        xsh_ping},
        {"ps",          xsh_ps},
        {"sleep",       xsh_sleep},
        {"tee",         xsh_tee},
        {"udp",         xsh_udpdump},
        {"udpecho",     xsh_udpecho},
        {"udpeserver",  xsh_udpeserver},
        {"uptime",      xsh_uptime},
        {"mycmd",       xsh_mycmd},
        {"namecmd",     xsh_namecmd},
        {"?",           xsh_help}

};

uint32  ncmd = sizeof(cmdtab) / sizeof(struct cmdent);


/************************************************************************/
/*                                                                      */
/* cmdlookup -- return the index in cmdtab of a command name            */
/*                                                                      */
/************************************************************************/

C ▾   Tab Width: 8 ▾   Ln 34, Col 38   INS
```

Lalu kita Deklarasi: Tambahkan ke shprototypes.h dan save

```
Activities  gedit  Mon 4 May, 22:52
Open  *shprototypes.h  ~/xinu/include  Save
extern  shellcmd  xsh_ping      (int32, char *[]);

/* in file xsh_ps.c */
extern  shellcmd  xsh_ps        (int32, char *[]);

/* in file xsh_sleep.c */
extern  shellcmd  xsh_sleep     (int32, char *[]);

/* in file xsh_udpdump.c */
extern  shellcmd  xsh_udpdump   (int32, char *[]);

/* in file xsh_udpecho.c */
extern  shellcmd  xsh_udpecho   (int32, char *[]);

/* in file xsh_udpeserver.c */
extern  shellcmd  xsh_udpeserver (int32, char *[]);

/* in file xsh_uptime.c */
extern  shellcmd  xsh_uptime    (int32, char *[]);

/* in file xsh_mycmd.c */
extern  shellcmd  xsh_mycmd     (int32, char *[]);

/* in file xsh_namecmd.c */
extern  shellcmd  xsh_namecmd   (int32, char *[]);

/* in file xsh_help.c */
extern  shellcmd  xsh_help      (int32, char *[]);
C/C++/ObjC Header  Tab Width: 8  Ln 82, Col 1  INS
```

Lalu kita kompilasi ulang dengan masuk kedirektori compile dan ketikkan perintah make clean dan make

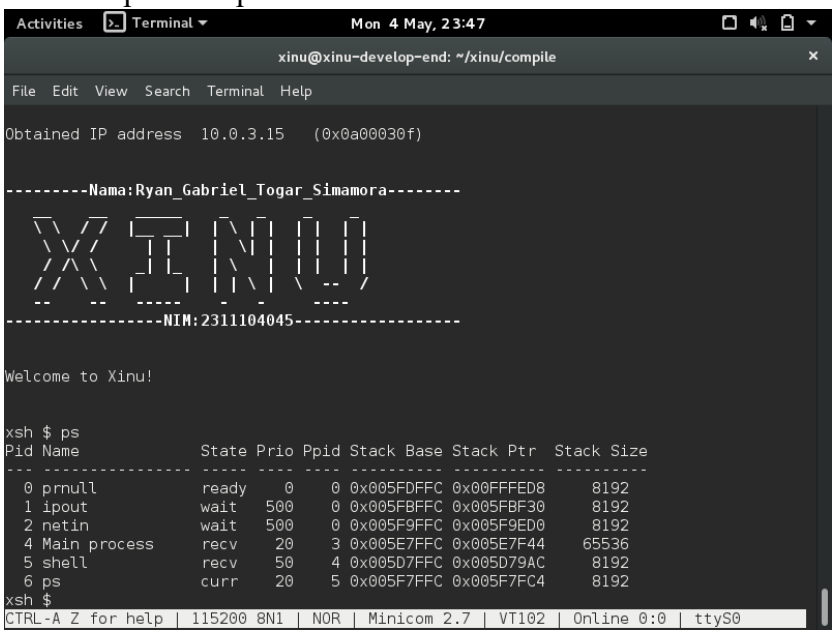
```
Activities  Terminal  Mon 4 May, 23:49
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$ sudo minicom
[sudo] password for xinu:
```

3. **[20 Poin]** Test hasilnya:

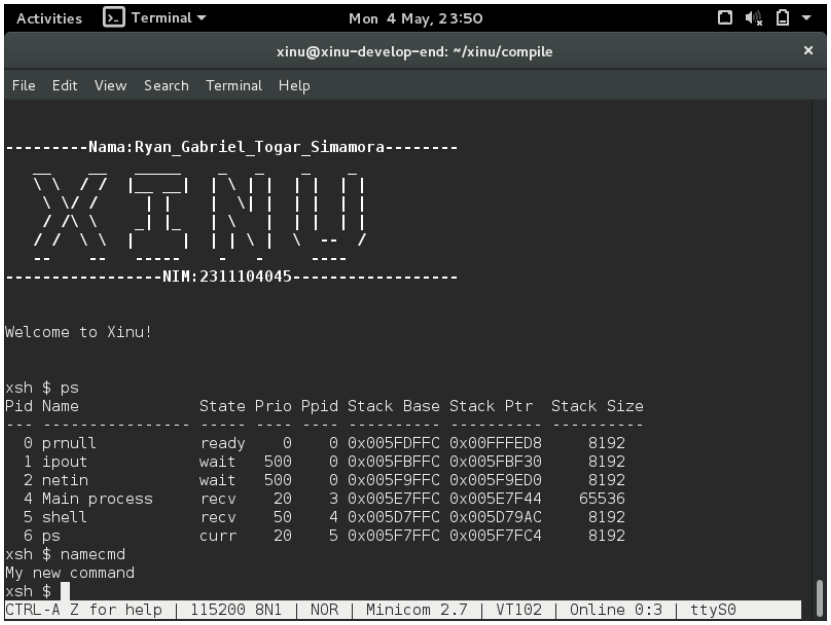
   a. Masuk ke terminal xinu
   b. Jalankan perintah ps
   c. Jalankan perintah namecmd
   d. Jalankan perintah ps
   e. Lihat nama proses telah berubah

Jawab :

Jalankan perintah ps



Jalankan perintah namecmd



Jalankan perintah ps

```
Activities  Terminal  Mon 4 May, 23:53
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
----------------NIM:2311104045-----------------


Welcome to Xinu!


xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFED8     8192
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  6 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $ namecmd
My new command
xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFEC0     8192
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  8 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:6 | ttyS0
```

Lihat nama proses telah berubah

Jawab : Proses yang berubah ada di bagian prnull dibagian stack ptr byte memorynya berubah yang awalnya 0x00FFFED8 menjadi 0x00FFFEC0

BAB III
PENUTUP

Kesimpulan

Pada akhir modul pekan 10 ini, kita telah mempelajari cara memahami dan memodifikasi Shell Xinu melalui penambahan perintah baru. Berikut ringkasannya:

Kita memahami fungsi Shell sebagai antarmuka utama antara pengguna dan kernel, bekerja dalam pola Read-Parse-Execute untuk menerjemahkan input teks menjadi instruksi sistem.
Kita mempelajari arsitektur Xinu di mana setiap perintah merupakan fungsi internal C yang terintegrasi dengan kernel, sehingga setiap modifikasi membutuhkan kompilasi ulang seluruh sistem.

Kita mempraktikkan proses penambahan perintah baru melalui registrasi di shell.c, deklarasi di shprototypes.h, serta implementasi pada xsh_mycmd.c, hingga pengujian pada terminal serial.